Fine-Tuning an Open-Source LLM Using LoRA / QLoRA

Project Brief • Daily Tasks • Deliverables • Evaluation

Duration	5 days
Level	Intermediate to Advanced
Domain	Generative AI / LLM Fine-Tuning
Use case	AI Technical Support Assistant

Objective

Fine-tune a small, open-source language model on a custom technical question-answering dataset, compare its performance with the original model, and expose the fine-tuned model through an inference API.

1. Project Overview

The intern will build an AI Technical Support Assistant that answers questions about Python, APIs, databases, and common software-development issues. The project focuses on the complete fine-tuning workflow: dataset preparation, baseline evaluation, parameter-efficient training, model comparison, API development, testing, and documentation.

Project Goals

- Prepare a clean instruction-tuning dataset from appropriately licensed and verified technical sources.
- Run the original pretrained model against a fixed set of test questions and record baseline outputs.
- Fine-tune the model using LoRA or QLoRA with Hugging Face tools.
- Compare the original and fine-tuned models using consistent evaluation data and criteria.
- Serve the fine-tuned model through a FastAPI inference endpoint.
- Document the implementation, results, limitations, and setup instructions.

Technology Stack

Technology	Purpose
Python	Primary programming language
Hugging Face Transformers	Load model/tokenizer and run training/inference
Hugging Face Datasets	Prepare and manage training/evaluation data
PEFT with LoRA/QLoRA	Parameter-efficient fine-tuning
PyTorch	Model training and inference backend
Google Colab GPU or suitable GPU machine	Training environment
FastAPI	Expose model inference through REST API
Pydantic	Validate API input and output structures

Scope and Constraints

- Choose a small, compatible instruction-tuned model based on available GPU memory, model license, and runtime.
- Use a small pilot training run before starting a longer run.
- Use parameter-efficient fine-tuning; full-parameter fine-tuning is out of scope.
- Keep the task limited to technical question answering.
- A production deployment, user interface, and training a model from scratch are out of scope.
- Fine-tuning is not assumed to improve performance; the intern must report measured results honestly.

2. Five-Day Live Schedule

Day	Focus	Expected Deliverable
Day 1	Dataset preparation and baseline evaluation	Dataset, preprocessing script, selected model rationale, baseline predictions
Day 2	LoRA/QLoRA fine-tuning	Training script/notebook, logs, hyperparameters, saved adapter
Day 3	Evaluation and model comparison	Evaluation script, comparison report, failure-case analysis
Day 4	Inference API development	FastAPI application, request/response examples, API test results
Day 5	Final testing and demonstration	Complete source code, README, evaluation report, final demonstration

3. Day 1 — Dataset Preparation and Baseline Evaluation

Objective: Understand the selected model, prepare the dataset, and measure the original model's baseline performance.

Tasks

- Select a suitable open-source instruction-tuned language model.
- Review the model card, architecture, license, limitations, and hardware requirements.
- Prepare a dataset of technical questions and verified answers.
- Convert examples into an appropriate instruction-tuning format.
- Split the data into training and validation sets; keep a separate held-out test set.
- Run the original model against at least 20 fixed test questions and save its outputs.
- Record baseline limitations and examples of incorrect or incomplete responses.

Example Dataset Record

```
{
  "instruction": "Explain the difference between GET and POST requests.",
  "input": "",
  "output": "GET requests retrieve information, while POST requests submit data for processing."
}
```

Deliverables

- Dataset file and preprocessing script.
- Selected model name and brief justification.
- Baseline predictions for at least 20 test questions.
- Short report describing the original model's limitations.

End-of-Day Review Questions

1. Why did you select this model?
2. Why do we need training, validation, and test datasets?
3. What is the difference between pretraining and fine-tuning?
4. What limitations did you observe in the original model?

Completion criteria: The dataset is ready, and the original model can generate reproducible baseline predictions.

4. Day 2 — Fine-Tuning with LoRA/QLoRA

Objective: Configure and execute a fine-tuning run using parameter-efficient training.

Tasks

- Load the selected model and tokenizer.
- Tokenize and format the training dataset according to the model's expected chat or instruction template.
- Configure training arguments and evaluation/checkpoint settings.
- Integrate LoRA or QLoRA through PEFT.
- Run a small pilot to check memory use and training behavior.
- Train the model and monitor training and validation loss where feasible.
- Save the trained adapter, tokenizer/configuration information, and training logs.

Concepts to Understand

- Tokenization and context length.
- Trainable versus frozen parameters.

- LoRA rank and target modules.
- Learning rate, batch size, and gradient accumulation.
- Training loss, validation loss, and overfitting.

Deliverables

- Fine-tuning script or notebook.
- Training logs and loss values/curves.
- Saved adapter and configuration.
- Record of model, dataset, hyperparameters, GPU type, and training duration.

End-of-Day Review Questions

5. Why did you use LoRA instead of full fine-tuning?
6. What is the purpose of the learning rate?
7. How does LoRA reduce training requirements?
8. How can you identify overfitting?

Completion criteria: At least one training run completes, and the resulting adapter can be saved. Training time depends on the model, dataset, and available hardware.

5. Day 3 — Evaluation and Model Comparison

Objective: Determine whether fine-tuning improved performance on held-out examples.

Tasks

- Load the original model and fine-tuned model/adapter.
- Evaluate both models on the same held-out test questions.
- Generate answers for at least 50 held-out questions if feasible; otherwise document the actual count.
- Compare outputs using a consistent rubric.
- Identify improvements, regressions, and failure cases.
- Measure inference latency and memory usage where practical.

Evaluation Metrics

Metric	Purpose / Notes
Training and validation loss	Monitor training behavior; not sufficient alone to prove answer quality.
Exact Match	Useful for short answers with a single expected form; limited for open-ended responses.
ROUGE-L	Measures text overlap with reference answers; should not be treated as a complete quality metric.
Human evaluation	Assess correctness, relevance, completeness, and clarity using the same rubric for both models.
Inference latency	Compare response generation time under comparable conditions.
Memory usage	Record resource requirements for inference.

Deliverables

- Evaluation script and test dataset.
- Baseline-versus-fine-tuned comparison report.
- Sample predictions from both models.
- Analysis of at least five failure cases.

- Conclusion supported by results, including cases where fine-tuning did not help.

End-of-Day Review Questions

9. Did fine-tuning improve the answers, and what evidence supports your conclusion?
10. How did you measure improvement?
11. Did performance regress on any questions?
12. What would you change in the dataset or training setup next?

Completion criteria: A reproducible comparison is produced. Improvement is desirable but is not a guaranteed completion requirement.

6. Day 4 — Build an Inference API

Objective: Make the fine-tuned model accessible through a REST API.

Tasks

- Load the fine-tuned adapter and its compatible base model.
- Implement a FastAPI application.
- Create a prediction endpoint.
- Validate inputs and handle empty or invalid requests.
- Add basic error handling and response-time logging.
- Test the endpoints using Postman or curl.

Required Endpoints

Endpoint	Method	Purpose
/health	GET	Check whether the application is running.
/predict	POST	Generate an answer using the fine-tuned model.

Example Request

```
{
  "question": "What is a Python decorator?"
}
```

Example Response

```
{
  "question": "What is a Python decorator?",
  "answer": "A Python decorator is a function that modifies or extends the behavior of another function or method.",
  "model": "fine-tuned-model"
}
```

The example response is illustrative; actual wording depends on the model.

Deliverables

- FastAPI application and model-loading/inference code.
- API request/response examples.
- Test results, including invalid-input cases.
- Instructions for running the API.

Completion criteria: A user can submit a question to the API and receive a response generated by the fine-tuned model.

7. Day 5 — Final Testing and Demonstration

Objective: Deliver a complete, documented, and evaluated fine-tuning project.

Tasks

- Run the workflow from dataset preparation through inference.
- Verify that the adapter can be loaded outside the training session.
- Test at least 20 representative API requests.
- Test empty inputs, invalid requests, and recoverable errors.
- Review performance against the baseline and report the results accurately.
- Complete the README and architecture/workflow explanation.
- Demonstrate the application and explain key implementation choices.

Final Submission Checklist

- Complete source code.
- Dataset and preprocessing scripts.
- Model and training configuration.
- Fine-tuning script or notebook.
- Saved model adapter (or documented artifact location if too large to submit).
- Evaluation results and comparison report.
- FastAPI application.
- Postman collection or equivalent API examples.
- README with setup and execution instructions.
- Short demonstration or recorded walkthrough.

Final Review Questions

13. What changed during fine-tuning?
14. Which parameters were trained?
15. Why did you choose LoRA or QLoRA?
16. How did you determine whether the model improved?
17. What limitations remain?
18. How would you support multiple concurrent users?
19. How would you update the model when new training data becomes available?

8. Live Supervision Plan

Day	Assignment	Review Before Moving On
Day 1	Dataset preparation and baseline evaluation	Dataset, model rationale, baseline predictions
Day 2	LoRA/QLoRA fine-tuning	Training logs, configuration, saved adapter
Day 3	Evaluation	Comparison report and failure analysis
Day 4	API development	Working endpoints and test results
Day 5	Final integration	Demonstration, source code, README, and final report

Supervisor guidance: Review the day's deliverables before assigning the next day's work. If the intern is blocked, ask for a short explanation of the blocker and the steps attempted. Keep the project scope fixed unless a technical constraint requires a documented adjustment.

9. Recommended Project Constraints

- Use a small, compatible pretrained model and an available GPU environment.
- Begin with a small, clean dataset and a short pilot run.
- Use LoRA/QLoRA rather than full-parameter fine-tuning.
- Keep the task limited to technical question answering.
- Do not require a UI or production deployment.
- Use a held-out test set that is not used for training or hyperparameter tuning.
- Record model version, dataset source, hyperparameters, hardware, and evaluation method.
- Do not claim that fine-tuning improved the model unless evaluation supports that conclusion.

10. Definition of Success

The project is complete when the intern can demonstrate a reproducible fine-tuning workflow, explain the training choices, load and serve the resulting adapter through an API, and provide an evidence-based comparison with the original model. The goal is sound engineering and learning—not a guaranteed performance improvement within five days.